

Research & Discovery

1. Executive Summary

- Brief summary of the context, history, and nature of your discovery effort. Is this net-new territory, or is there prior art?
- What's the goal of this effort, and on what timeline?
- How will this impact the company, our customers, and the ecosystem more broadly?

2. Opportunity Statements

- What discrete opportunities are in scope? Include qualitative and quantitative impact detail.

3. Market & Competitive Context

- What's happening in market related to this problem space?
- What are competitors doing here? How is that relevant to this discovery effort?

4. Target Personas & User Needs

List relevant user personas and outline their needs/challenges.

a. [User Persona Name]

- *Goals*: This user's goals
- *Pain Points*: What impedes the user from accomplishing their goals?
- *Opportunities*: How might we help?
- *Success*: What would success look like for this user?

5. Strategic Objectives & Success Metrics

Outline your discovery objectives. Use SMART goals. What are you...

- Researching...
- Documenting...
- Designing...
- Validating...
- Planning...
- Deciding...

6. Solution Overview

7. Adoption & Rollout Plan

Phase 1: Pilot (Weeks 1–4)

- Limited rollout to 2 manager users + 5 processors - 1 site
- Daily feedback loop and usage tracking

Phase 2: Broader Ops Rollout (Weeks 5–8)

- Roll out to 3–5 additional sites with different workflows
- Include training and enablement guides for huddle facilitation

Phase 3: Full Rollout + GA (Weeks 9–12)

- Launch to all users
- Measure impact on KPIs
- Begin sunsetting legacy landing page

9. Risks & Mitigations

Risk	Mitigation
Misidentification of user role or workflow needs	Use research and field testing with real users
Overwhelming users with too much data	Prioritize simplicity and only show key actionable elements
Resistance to change	Involve early adopters in design; provide side-by-side view of old vs. new
Backend data latency or inconsistency	Establish SLAs or weave in performance improvement work

10. Discovery Plan

Phase 1: Research

- Part 1
 - POVs from Stakeholders (Geoff, Tony F, Doug, Amy R, Zone leads)

- Part 2
 - User Research
 - Competitive Analysis (Internal and External)
- **Deliverable:**
 - Job map of current workflow (high level lifecycle of request (prioritized, assigned, processed, tracked)
 - Decision Tree of how requests are prioritized
 - Sized / Clear Problem Statements

Phase 2: Problem Refinement / Prioritization

- Understanding of JTBD (Persona based)
- **Deliverable:**
 - Crafting potential solutions
 - North Star (Customer Value)
 - Prioritization (Impact Estimate)

Phase 3: Solution Design

- Review above and evaluate solutions (internal or external)
- **Deliverable:**
 - Build / Buy / Partner analysis
 - Design concepts
 - Validate concepts

Phase 4: Roadmapping

- Sizing problems to be solved - eng effort
- **Deliverable:**
 - Technical Design
 - Stakeholder buy-in / Feedback
 - Roadmap - estimates on delivery

Product Spec

Clear Opportunity Statement

What opportunities are you solving for? Why is this important (business/customer impact)?

- Include baseline metrics to clarify the impact and priority of the issue.
-

Prioritized Scope

- Clearly define what is **in scope**.
 - List any **non-conditions** or **out-of-scope** items.
-

Acceptance Criteria

- Define clear, testable conditions for success.
-

Feature Type

Select one of the following:

- **Bug** – A defect (not a design/requirements issue).
- **Operational Change** – Routine operational need, e.g., a script to update records in bulk.
 - Note: Significant complexity or risk = treat as feature.
- **Small Enhancement** – Modest impact, ≤ 1 engineer sprint, minimal dependencies.
- **Feature** – Substantial change, ≥ 1 engineer sprint, complexity or dependencies, stakeholder sign-off.
- **Major Feature** – Extraordinary impact/effort/complexity. Usually the most important release feature.

User-Facing?

Does the feature change any of the following:

- User workflows
- User interfaces
- User-facing copy
- UI components
- Service or operations workflows

All user-facing changes require design.

Links to Context and External Documentation

- Research
 - Message threads
 - Decision notes
 - Detailed product requirements
-

Designs / Prototypes

- Links to design materials, wireframes, or finished designs
 - Include validation and user testing results
 - Call out any design system gaps that affect sizing
-

Dependencies & Constraints

- List tech/product teams you'll depend on (e.g., engineering teams, platform requirements)
 - Include non-technical dependencies (e.g., operational support, training)
-

Risks & Mitigation Strategies

For each known or potential risk:

- Describe the risk
 - Estimate probability and impact
 - List mitigation strategies and responsible owners
-

Data & Analytics Plan

- Inputs, outputs, and tracking needs
 - Data model changes
 - How will feature adoption, impact, and customer value be measured?
-

Open Questions

- List unresolved questions
 - Record answers as discovery progresses
-

Stakeholder Sign-Off Matrix

- Document required reviews

- Note completed sign-offs

Tech Spec

Purpose

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we'll balance many competing priorities to achieve a solution that delivers optimal business value.

Section Owner:

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

Order of Operations

The product and engineering teams collaborate on this document to provide increasing levels of clarity around the requirements and best approach that provides the most business value for each situation.

This process will take several distinct tickets and meetings. The goal is to provide initial visibility to the business about risks, opportunities, and costs. As we hone in on identifying exactly what we build and how much time we have available, the later stages involve finalizing the actual technical design that will be built.

While the whole document is needed for a complete review, a "first pass" sizing should always include:

- Design Goals
- One or more proposed architectural designs
- Size Estimate
- Confidence
- Risks/Mitigations table
- Rollout mechanics

Subsequent passes should increase the detail level (accuracy and precision) of the above sections.

During the final pass discovery stage and prior to the kickoff meeting, all remaining sections should be completed as best as possible.

Note: the kickoff meeting must always include Tyler Beach to help satisfy requirements for CASTLE

(<https://datavant.atlassian.net/wiki/spaces/PS/pages/1508179991/CASTLE+for+Engineers>)

Please also include Danielle Sill on all Kickoff meetings to bring visibility to her and her team.

System Context

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.

What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

Design Goals

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.

What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

Proposed Architectural Designs

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.

What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

Size Estimate

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.

What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Confidence

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.

What: One or two sentences describing how confident we are about our size estimate.

Risks/Mitigations

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.

What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	Mitigation

Rollout Mechanics

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.

What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout

the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.

If rollback is for some reason not possible in a step, note that here so we can understand the implications.

This section is intended to be complementary to the **Release Strategy** tab.

Security Considerations and Implications

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.

What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

Monitoring, Alerting, and Measuring

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.

What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

Testing

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.

What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Dependencies

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.

What: A bullet point list of external dependencies that we will need in order for this project to be successful.

Release Strategy

Release Strategy & Iterations

Maturity Level (for Feature or Major Feature):

- **Experiment** – Small-scale (<5 customers) for early hypothesis testing
- **Alpha** – Initial test with up to 10 customers
- **Beta** – Scaled test with up to 100 customers
- **General Availability (GA)** – Broad release to all customers

Normally, features progress through alpha → beta → GA. Skipping steps requires justification.

Rollout Toggles

- Indicate whether a toggle is needed
- PMs should use toggles unless clearly unnecessary
- Toggles, when needed, should ideally be labeled with ticket numbers so that complexity can be manageable long term for our future selves.

Iterations

- Define phases (alpha, beta, GA)
- Include rough timelines
- Note scope and exclusions for each stage